# Supplementary Material for DistCM-IL: A post-clustering tool for improved edge-connectivity

Haotian Yi[1], Charles Cooper[1], Minhyuk Park[1], Tandy Warnow[1*], George Chacko[1*]

[1]Siebel School of Computing and Data Science, The University of Illinois Urbana-Champaign, 201 N. Goodwin Ave, Urbana, 61801, Illinois, USA.

*Corresponding author(s). E-mail(s): warnow@illinois.edu; chackoge@illinois.edu;
Contributing authors: yi54@illinois.edu; ccoop4@illinois.edu; minhyuk2illinois.edu;

## 1 DistCM-IL Design & Implementation Details

This section describes some additional design and implementation details of DistCM-IL.

### 1.1 Partitioning and Batching

Before execution, DistCM-IL makes a sequential pass over the input that partitions the network into self-contained jobs on the shared file system, so that each worker loads only the clusters it processes. With one job per cluster, a clustering with millions of small clusters pays a file-creation cost in this pass, and a head-worker round trip during execution, for every cluster. DistCM-IL therefore groups small clusters into a single job until their combined estimated cost $Y(c)$ reaches a user-specified threshold, which we call a batch. The batch is then stored, assigned, and processed as a unit.

Table 1 shows the effect on OCv1. Enabling batching at a threshold of 1,000 reduces partitioning time by 1.7× to 7.8×, with the largest gain for the clustering with the most clusters. Raising the threshold further changes partitioning time by 10-22%, in no consistent direction, while leaving the head node fewer and coarser jobs to balance. We therefore set the default threshold to 1,000.

**Table 1**: **Effect of batching on Dist-CM partitioning** - Results are taken from network OCv1. Without batching ("None"), every cluster is its own job, so the number of batches equals the number of input clusters. Runtimes and batch counts are means over 3 replicates. Singleton clusters are excluded.

| | Leiden-Mod | | Leiden-CPM(0.001) | | Leiden-CPM(0.01) | |
|---|---|---|---|---|---|---|
| Min. batch cost | Time (s) | Batches | Time (s) | Batches | Time (s) | Batches |
| None | 1,147.3 | 183,432 | 1,803.3 | 840,737 | 3,710.3 | 2,133,756 |
| 1,000 | 675.0 | 1,469 | 514.0 | 88,939 | 473.3 | 149,153 |
| 10,000 | 674.3 | 195 | 625.7 | 14,265 | 389.3 | 16,349 |
| 100,000 | 604.0 | 67 | 609.7 | 1,557 | 374.3 | 1,653 |

## 1.2 Work Sharing

To avoid a large cluster from bottlenecking a worker, DistCM-IL allows subsequent tasks to be shared with other workers as the cluster is decomposed. When a CM iteration produces more than one sub-cluster above a user-specified size threshold, the worker keeps the largest and yields the others to the head node, which inserts them into its priority queue like other assignable jobs. Sub-clusters smaller than the threshold are kept locally to reduce communication overhead. The mechanism is recursive in nature: a worker that receives a shared sub-cluster may further share parts of it, so a large cluster fans out into a tree of progressively smaller jobs spread across many workers (Figure. 1).

Since work is now split across this tree, completion must be tracked hierarchically. The head node marks a cluster complete only once every sub-cluster yielded from it has completed. Only then is every node of the original cluster guaranteed to have been processed by the post-processing algorithm and counted toward overall progress.

Yielding a sub-cluster requires writing its edgelist to the shared file system and reading it back on another worker, so each redistribution incurs I/O proportional to the sub-cluster's size. The threshold therefore controls a trade-off: a low threshold redistributes aggressively, maximizing parallelism but multiplying I/O, whereas a high threshold avoids communication at the risk of letting a large cluster to bottleneck a single worker. We use a default of 100,000 nodes, chosen so that the time saved by processing a sub-cluster in parallel outweighs the I/O cost of moving it.

## 1.3 Termination Logic

After all initial clusters are either completed or aborted, the program enters the termination phase, in which results are aggregated into the final output. Since each assignment produces a self-contained result, the final clustering is constructed by combining the results of each individual cluster. DistCM-IL implements a simplification of this classic reduction process with two levels, where each worker first merges the results of all completed clusters it worked on into a single local output, before the head node merges the outputs of all workers into the final clustering. This two-level scheme could be generalized to a tree-structured reduction with $O(\log N)$ complexity using $N$

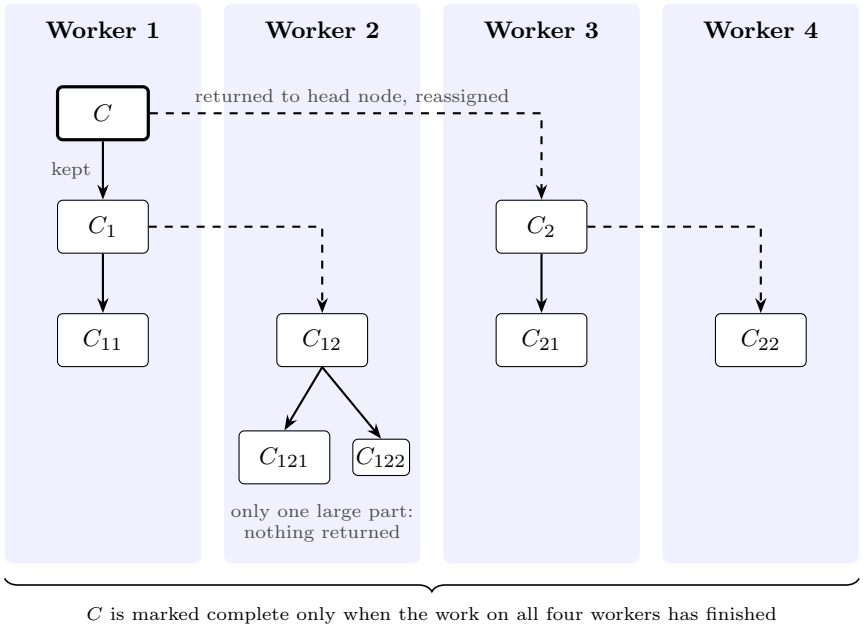



**Fig. 1**: **Recursive work sharing** - Each box is a cluster and each column is the worker that processes it. When a split produces more than one sub-cluster above the size threshold, the worker keeps one (solid arrows) and returns the others to the head node, which reassigns them to idle workers (dashed arrows). A reassigned sub-cluster can be split and returned again ($C_2$), whereas a split with only one large part stays on the same worker ($C_{12}$). The head node marks $C$ complete only when all work derived from it has finished. If any part fails, $C$ is marked aborted.

workers, but in practice the termination phase takes negligible time compared to the actual processing, so we keep this simplified implementation.

## 1.4 Fault Tolerance

Because the induced subgraphs at each level of recursion are independent, a failure while processing one cluster cannot affect the others. DistCM-IL exploits this independence through a checkpointing mechanism. The system preserves progress and allows execution to quickly resume after individual cluster failures.

Progress is recorded at the granularity of initial clusters. When a cluster fails - whether from timeout, out-of-memory, or other faults - it is marked incomplete and written to the checkpoint file; if any descendant sub-cluster produced from it by work-sharing fails, the entire initial cluster is likewise marked incomplete. On resumption, the head node loads the checkpoint and assigns only the incomplete clusters. This coarse granularity trades some redundant recomputation for simplicity and correctness: it avoids the machinery needed to track and validate the intermediate state of

partially completed clusters, which could otherwise leave the system corrupted. Finer-grained checkpointing that recovers partial progress within a cluster is a promising direction for further improving efficiency on extreme-scale networks.